The PDHG Algorithm used in Computerised Tomography

MATH40011

Author: Jan Exley

Supervisor: Dr. Sean Holman

Department of Mathematics
University of Manchester

January 6, 2026

Contents

1 Introduction **2**

2 Theory **3**
- 2.1 Radon Transform . . . 3
  - 2.1.1 So what does the Radon Transform look like? . . . 4
  - 2.1.2 Parametrization of curve $L_{(\theta,s)}$ . . . 4
  - 2.1.3 Putting all together . . . 5
- 2.2 Inverse of the Radon Transform . . . 7
  - 2.2.1 Fourier Slice Theorem Proof . . . 7
  - 2.2.2 Filtered Back Projection Method (FBP) . . . 9

3 Numerical Algorithms **10**
- 3.1 Gradient Descent using Least Squares Algorithm . . . 12
- 3.2 Adding Regularisation . . . 13
- 3.3 Proximal Maps . . . 16
- 3.4 Primal & Dual Problem . . . 18
- 3.5 PDHG Algorithm . . . 21

4 Conclusion **24**

A Appendix **25**
- A.1 Convergence Proof . . . 25
- A.2 Dual Problem Formula Derivation . . . 25
- A.3 Relation Between infA and supA Proof . . . 26

References **27**

Abstract

Image reconstruction in computerised tomography (CT) is a classical inverse problem in which an object is reconstructed from its line integral measurements. While the inverse Radon transform admits analytical solutions in idealised continuous settings, practical CT systems produce discrete and noisy data, requiring numerical reconstruction algorithms.

This project develops the mathematical framework underlying CT image reconstruction, beginning with the analytical formulation of the Radon transform and its inversion via filtered back projection. The limitations of analytical methods in realistic CT settings are then discussed, motivating the use of optimisation-based numerical techniques. In particular, the project focuses on the Primal–Dual Hybrid Gradient (PDHG) algorithm. Deriving and building it up from other algorithms, and demonstrating the role of stabilised CT reconstruction when adding regularisation.

1 Introduction

This whole project is under the broader topic of inverse problems. A type of problem where you start with the results and try to determine the causes that produced them. This is in contrast to what most undergraduate degree courses study, a forward problem, which starts with known causes (a function $f(x)$) and computes the effects ($y(= f(x))$). If the forward problem can be written as

$$y = f(x),$$

where x are the inputs and y is the measured data, then the inverse problem is,

$$\text{find } x \text{ such that } f(x) = y_{obs},$$

where y_{obs} is your observed data.

In this project we will look at an example of an inverse problem, computerised tomography (CT). Mathematically, the CT problem consists of recovering a function, $f(x, y)$, from its integrals over a set of lines. This mapping from the function to its line integrals is known as the Radon transform $\mathcal{R}$,

$$f(x, y) \mapsto \mathcal{R}[f](\theta, s) = \int_{L_{(\theta,s)}} f(x, y)dl,$$

where $L_{(\theta,s)}$ is the line with normal direction θ at a signed distance s from the origin. Image reconstruction in CT corresponds to inverting this transform. It is this inverse Radon transform problem that is traditionally referred to as tomography in mathematical literature. And there are many ways to do the inverse and find f, both analytical and numerical.

What makes CT computerised is the implementation of measurements using detectors and other similar devices all working in unison. These devices require the use of numerical algorithms to reconstruct $f(x, y)$, as the function

$g(\theta, s) = \mathcal{R}[f](\theta, s)$ is continuous and cannot be measured exactly in practice. A CT scanner can only acquire a finite set of discrete samples of $g(\theta, s)$, not a continuous set of samples of $g(\theta, s)$. The CT scanner will take discrete samples of intensity, after beams pass through the object f, at many angles θ_i and offsets s_j. These samples form a sinogram, a digital dataset representing $g(\theta_i, s_j)$. So instead of a continuous transform, we get a discrete one:

$$g_{ij} = \mathcal{R}[f(\theta_i, s_j)] + \text{noise}.$$

Therefore the inversion of the Radon transform (which is continuous and integral-based) is replaced by numerical algorithms to reconstruct $f(x, y)$ from the sampled data g_{ij}.

One of these reconstruction algorithms that can be implemented numerically is the Filtered Back Projection (FBP) method. This project goes over this methods analytical formulation, see Section 2.2.2.

But sometimes that noise described in the equation above is not small, or the measured data $g(\theta, s)$ is undersampled, meaning that only a limited number of projection angles θ_i are available. In that case we require numerical optimization algorithms to help get the best possible reconstruction of our data, and we may wish to use algorithms such as the Primal–Dual Hybrid Gradient (PDHG) algorithm instead of FBP. It's the PDHG algorithm that we will be focusing on in this project.

2 Theory

The Theory Section outlines the Maths involved to solve the CT problem, as mentioned in the introduction 1. In particular, we answer what is the Radon transform. And address inverting the Radon transform and hence solving the CT problem.

2.1 Radon Transform

In the introduction 1, we mentioned the Radon Transform. The Radon Transform is a linear operator:

$$\mathcal{R} : C_c^\infty(\mathbb{R}^2) \to C^\infty([0, \pi) \times \mathbb{R}),$$

where $C_c^\infty(\mathbb{R}^2)$ denotes the space of smooth functions with compact support. For $f \in C_c^\infty(\mathbb{R}^2)$, the Radon transform $R[f](\theta, s)$ is defined as the integral of f over the line with normal direction θ at signed distance s from the origin.

Although this project primarily works with discretised data, the Radon transform can be extended by continuity to larger function spaces, such as $L^2(\mathbb{R}^2)$, which are more relevant for practical CT reconstruction.

There are many examples of linear operators, for example Matrix multiplication (eg. $\mathcal{R}(\boldsymbol{x}) = A\boldsymbol{x}$). In this case, we are dealing with an integral operator, as was hinted at in the introduction 1. An integral operator takes the function

$f(x, y)$, operates on it using some form of integration, and produces a function $g(\theta, s)$:

$$\mathcal{R}[f](\theta, s) = g(\theta, s),$$

ie. $f(x, y) \mapsto g(\theta, s) = \mathcal{R}[f](\theta, s)$.

2.1.1 So what does the Radon Transform look like?

The Radon Transform is named after Johann Radon, who in his paper in 1917 [1], translated from German [2]. Proved that an object, $f(x, y)$, can be reconstructed perfectly from a full set of line integrals over all angles.

A line integral, is a type of integral where you integrate a function along a curve rather than over an interval or area. If we integrate the function $f(x, y)$ over the line $L_{(\theta,s)}$, which is defined by parameters s and θ, the resulting line integral would be written as,

$$\mathcal{R}[f](\theta, s) = \int_{L_{(\theta,s)}} f(x, y) dl.$$

where $\theta \in [0, \pi)$ specifies the normal direction of the line and $s \in \mathbb{R}$ denotes its signed distance from the origin.

To compute the line integral, we need to parametrise the curve, $L_{(\theta,s)}$ and the function, $f(x, y)$. See Section 2.1.2, for the parametrisation.

The Radon transform is obtained by evaluating these line integrals over all values of θ and s, so that the collection of lines $\{L_{(\theta,s)}\}$ covers the support of the function. Varying both parameters ensures that all relevant line integrals are included in the transform.

2.1.2 Parametrization of curve $L_{(\theta,s)}$

For the parametrisation, we choose the most simple curve, a straight line. The most suitable parametrization for this line is to consider the 2D polar-coordinate basis vectors $\{\hat{\boldsymbol{r}}, \hat{\boldsymbol{\theta}}\}$, which are

$$\hat{\boldsymbol{r}} = \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix}, \qquad \hat{\boldsymbol{\theta}} = \begin{pmatrix} -\sin\theta \\ \cos\theta \end{pmatrix}.$$

This allows us to specify the normal direction of this line $\theta \in [0, \pi)$, as can be seen in Figure 1. We can then write the straight line, $L_{(\theta,s)}$, in terms of these basis vectors,

$$\{\boldsymbol{x}(s) = s\hat{\boldsymbol{r}} + l\hat{\boldsymbol{\theta}} \ : \ l \in \mathbb{R}\},$$

where $s \in \mathbb{R}$, is the perpendicular distance from the origin in the (x, y) plane. And l is the parameter along the line. We can manipulate $\boldsymbol{x}(s)$, such that we eliminate l from the equation of a straight line $L_{(\theta,s)}$:

$$\boldsymbol{x} \cdot \hat{\boldsymbol{r}} = s(\cancelto{1}{\hat{\boldsymbol{r}} \cdot \hat{\boldsymbol{r}}}) + l(\cancelto{0}{\hat{\boldsymbol{\theta}} \cdot \hat{\boldsymbol{r}}}),$$

$$\boldsymbol{x} \cdot \hat{\boldsymbol{r}} = s,$$
$$\implies \{\boldsymbol{x} \cdot \hat{\boldsymbol{r}}(\theta) = s \ : \ \theta \in [0, \pi), s \in \mathbb{R}\}.$$

$[0, \pi)$, was chosen due to rotational symmetry of straight lines rotated by an angle of 180°, ie. $L_{(\theta,s)} = L_{(\theta+\pi,s)}$.

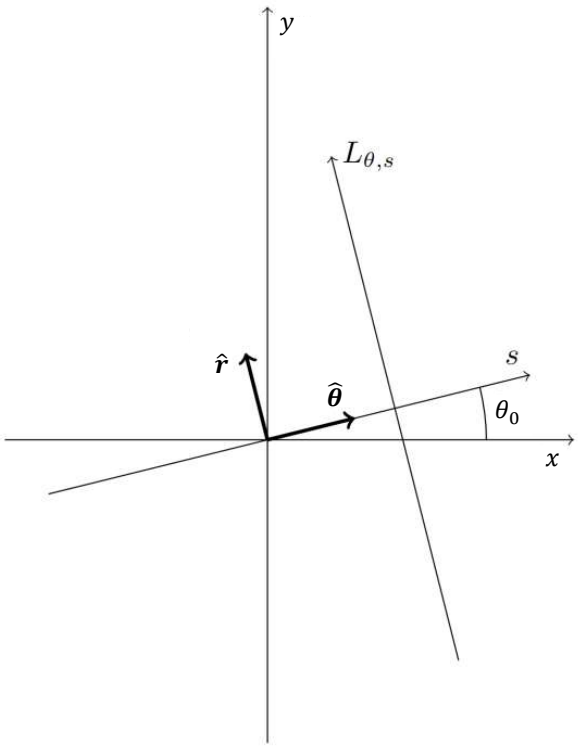


Figure 1: A visual representation of the basis vectors $\{\hat{\boldsymbol{r}}, \hat{\boldsymbol{\theta}}\}$ and the parameters s and θ_0, where θ_0 is fixed. Adapted from [3].

An extra mathematical equation that will come in handy later in Section 2.2.1, follows the same maths above and gets,

$$\boldsymbol{x} \cdot \hat{\boldsymbol{\theta}} = s(\cancelto{0}{\hat{\boldsymbol{r}} \cdot \hat{\boldsymbol{\theta}}}) + l(\cancelto{1}{\hat{\boldsymbol{\theta}} \cdot \hat{\boldsymbol{\theta}}}),$$
$$\boldsymbol{x} \cdot \hat{\boldsymbol{\theta}} = l.$$

2.1.3 Putting all together

The Radon Transform is just a line integral, expressed in terms of (θ, s):

$$\mathcal{R}[f](\theta, s) = g(\theta, s) = \int_{L_{(\theta,s)}} f(x, y) dl.$$

To actually calculate the integral, let's use our parametrisation from Section 2.1.2,

$$\boldsymbol{x} = \begin{pmatrix} x(l) \\ y(l) \end{pmatrix} = s \begin{pmatrix} \cos\theta \\ \sin\theta \end{pmatrix} + l \begin{pmatrix} -\sin\theta \\ \cos\theta \end{pmatrix},$$

so the integral becomes,

$$g(\theta, s) = \int_{-\infty}^{\infty} f(x(l), y(l)) dl$$

$$= \int_{-\infty}^{\infty} f(s\cos\theta - l\sin\theta, s\sin\theta + l\cos\theta) dl,$$

for some (θ, s).

If you calculate $g(\theta, s)$ for all (θ, s)-values, then Radon's theorem states that you can reconstruct $f(x, y)$ from $g(\theta, s)$. See Figure 2, to illustrate what $g(\theta, s)$ looks like for just one fixed angle θ, when calculated for some random shapes.

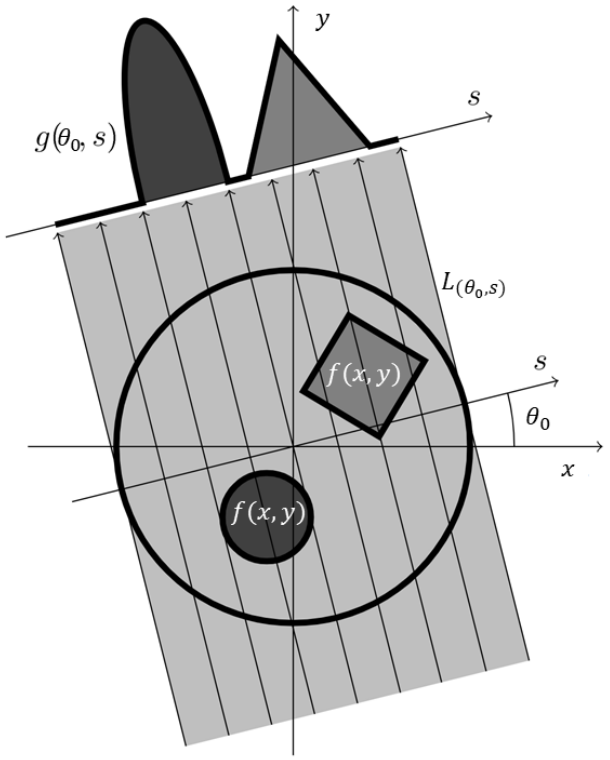


Figure 2: A visual example of how the Radon transform $g(\theta_0, s)$, for a fixed angle θ_0, is related to the image $f(x, y)$. Adapted from [3].

When the Radon transform $g(\theta, s)$ is evaluated over a range of angles and offsets, the resulting data can be visualised as an image known as a sinogram.

The sinogram is an intensity image of $g(\theta, s)$. It is called a sinogram as it typically exhibits sinusoidal patterns. An example of a sinogram is shown in Figure 3, which displays the object $f(x, y)$, given by the Shepp-Logan phantom used to generate its corresponding sinogram. The Shepp–Logan phantom is a standard test image in CT, consisting of a superposition of ellipses designed to mimic cross-sections of the human head.

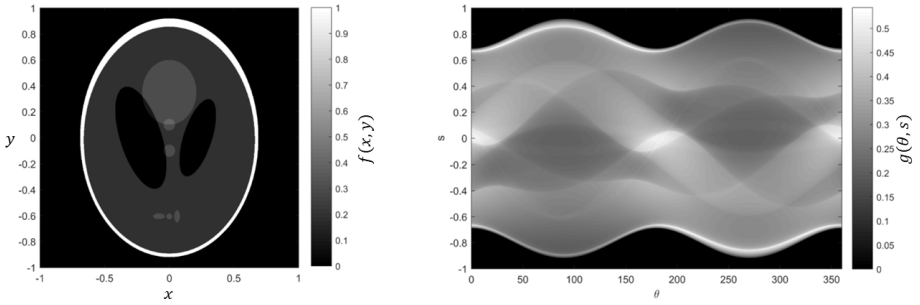


Figure 3: The Shepp–Logan phantom (*left figure*) used to generate the sinogram (*right figure*). Adapted from [3].

2.2 Inverse of the Radon Transform

Our aim is to find $f(x, y)$, the function that represents our internal structure from a CT scan. CT scans produce measured data $g(\theta, s)$, which is related to $f(x, y)$ by the Radon transform $\mathcal{R}$, through the equation $g(\theta, s) = \mathcal{R}[f]$.

So we need to find the operation that recovers $f(x, y)$ from this equation. This suggests the use of an inverse of the Radon Transform, $f(x, y) = \mathcal{R}^{-1}[g]$, where

$$\mathcal{R}^{-1}\mathcal{R}[f] = \mathcal{R}^{-1}[\mathcal{R}[f](\theta, s)] = \mathcal{R}^{-1}[g(\theta, s)] = f(x, y).$$

One might think that as the Radon transform is an integral operator, the inverse would be a differential operator. However, the inverse is more subtle and requires Fourier analysis, as we will see in the following proofs.

2.2.1 Fourier Slice Theorem Proof

To complete Filtered Back Projection, a method to invert the Radon Transform, we require the Fourier Slice Theorem. This Theorem relates the 1D Fourier transform of the Radon transform, $\widehat{\mathcal{R}[f]}$, with the 2D Fourier transform of our desired function $\widehat{f}(r\hat{\boldsymbol{r}})$.

Theorem 2.1 (Fourier Slice Theorem) *Let $f \in C_c^\infty(\mathbb{R}^2)$ and let $g(\theta, s) = R[f](\theta, s)$ denote its Radon transform. Then the one-dimensional Fourier transform of g with respect to s satisfies*

$$\widehat{R[f]}(r, \theta) = \sqrt{2\pi}\, \hat{f}(r\hat{\boldsymbol{r}}),$$

where $\hat{f}$ denotes the two-dimensional Fourier transform of f, and $\hat{\mathbf{r}} = (\cos\theta, \sin\theta)$.

To prove this theorem and the subsequent Filtered Back Projection method, we will need the Fourier transforms. As such we will use the following definitions and conventions for the 1D Fourier transform, 2D Fourier transform and the 2D inverse Fourier transform.

Definition 2.1 (1D Fourier transform) *Let $h \in L^1(\mathbb{R})$. The one-dimensional Fourier transform of h is defined by,*

$$\mathcal{F}[h](r) = \hat{h}(r) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} h(s)\, e^{-irs}\, ds,$$

where $r \in \mathbb{R}$.

Definition 2.2 (2D Fourier transform) *Let $h \in L^1(\mathbb{R}^2)$. The two-dimensional Fourier transform of h is defined by,*

$$\mathcal{F}[h](\boldsymbol{\omega}) = \hat{h}(\boldsymbol{\omega}) = \frac{1}{2\pi} \int_{\mathbb{R}^2} h(\boldsymbol{x})\, e^{-i\boldsymbol{\omega}\cdot\boldsymbol{x}}\, d\boldsymbol{x},$$

where $\boldsymbol{x} = (x, y) \in \mathbb{R}^2$ and $\boldsymbol{\omega} = (\omega_1, \omega_2) \in \mathbb{R}^2$.

Definition 2.3 (2D inverse Fourier transform) *Let $\hat{h} \in L^1(\mathbb{R}^2)$. The two-dimensional inverse Fourier transform of $\hat{h}$ is defined by,*

$$\mathcal{F}^{-1}[\hat{h}](\boldsymbol{x}) = h(\boldsymbol{x}) = \frac{1}{2\pi} \int_{\mathbb{R}^2} \hat{h}(\boldsymbol{\omega})\, e^{i\boldsymbol{\omega}\cdot\boldsymbol{x}}\, d\boldsymbol{\omega},$$

where $\boldsymbol{x} = (x, y) \in \mathbb{R}^2$ and $\boldsymbol{\omega} = (\omega_1, \omega_2) \in \mathbb{R}^2$.

We start the proof with the parametrised form of the Radon Transform as we defined in Section 2.1.3,

$$\mathcal{R}[f](s, \theta) = \int_{-\infty}^{\infty} f(x(l), y(l))dl = \int_{-\infty}^{\infty} f(s\cos\theta - l\sin\theta, s\sin\theta + l\cos\theta)dl.$$

Now take the 1D Fourier transform 2.1 with respect to s of the parametrised Radon transform to get,

$$\widehat{\mathcal{R}[f]}(r, \theta) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{-isr} \mathcal{R}[f](\theta, s)ds$$

$$= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{-isr} \int_{-\infty}^{\infty} f(s\cos\theta - l\sin\theta, s\sin\theta + l\cos\theta)dlds.$$

Now re-write everything back in terms of an integral with respect to x and y. Where $(x, y) \in \mathbb{R}^2$ are the Cartesian coordinates in the image domain. As before to make a change of variable, we need to calculate the determinant of

the Jacobian $\mathcal{J}$. To do this first calculate $l = l(x, y)$ and $s = s(x, y)$ for fixed θ, something we did in Section 2.1.2,

$$l = \boldsymbol{x} \cdot \hat{\boldsymbol{\theta}} = x \sin\theta - y \cos\theta \quad \& \quad s = \boldsymbol{x} \cdot \hat{\boldsymbol{r}} = x \cos\theta + y \sin\theta,$$

$$|\mathcal{J}| = |\frac{\partial(l, s)}{\partial(x, y)}| = \begin{vmatrix} \frac{\partial l}{\partial x} & \frac{\partial l}{\partial y} \\ \frac{\partial s}{\partial x} & \frac{\partial s}{\partial y} \end{vmatrix} = \begin{vmatrix} \sin\theta & -\cos\theta \\ \cos\theta & \sin\theta \end{vmatrix} = \sin^2\theta + \cos^2\theta = 1.$$

Hence, the integral becomes,

$$= \frac{1}{\sqrt{2\pi}} \int_{-\infty}^{\infty} e^{-i(x\cos\theta + y\sin\theta)r} \int_{-\infty}^{\infty} f(x, y) \cancelto{1}{|\mathcal{J}|} dx dy.$$

Noticing that everything can be written in vector form ie. $f(x, y) = f(\boldsymbol{x})$, we get

$$= \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}^2} e^{-i(\boldsymbol{x}\cdot\hat{\boldsymbol{r}})r} f(\boldsymbol{x}) d\boldsymbol{x} = \frac{1}{\sqrt{2\pi}} \int_{\mathbb{R}^2} e^{-i\boldsymbol{x}\cdot(r\hat{\boldsymbol{r}})} f(\boldsymbol{x}) d\boldsymbol{x}.$$

Notice that this is the 2D Fourier transform 2.2 of $r\hat{\boldsymbol{r}}$,

$$= \sqrt{2\pi}(\frac{1}{2\pi} \int_{\mathbb{R}^2} f(\boldsymbol{x}) e^{-i(r\hat{\boldsymbol{r}})\cdot\boldsymbol{x}} d\boldsymbol{x}) = \sqrt{2\pi}\widehat{f}(r\hat{\boldsymbol{r}}).$$

This concludes the proof as we have now proven the original equation.

2.2.2 Filtered Back Projection Method (FBP)

We now present an explicit inversion formula for the Radon transform, which allows the reconstruction of $f(x, y)$ from its projections $g(\theta, s) = R[f](\theta, s)$. This formula is commonly referred to as the filtered back projection (FBP) formula in the CT literature and is derived using Fourier analysis.

Theorem 2.2 (Filtered back projection) *Let $f \in C_c^\infty(\mathbb{R}^2)$ and let $g(\theta, s) = R[f](\theta, s)$ denote its Radon transform. Then f can be recovered from g via the formula:*

$$f(x, y) = \frac{1}{2\pi\sqrt{2\pi}} \int_0^{2\pi} \int_0^{\infty} \hat{g}(r, \theta)\, e^{ir\hat{\boldsymbol{r}}\cdot\boldsymbol{x}}\, r\, dr\, d\theta,$$

where $\hat{g}(r, \theta)$ denotes the one-dimensional Fourier transform of $g(\theta, s)$ with respect to s.

To prove Theorem 2.2, start by using the same equation $\mathcal{R}^{-1}\mathcal{R}[f] = f$ from Section 2.2. But instead of the Radon transform $\mathcal{R}$, use the Fourier transform $\mathcal{F}$,

$$f(x, y) = \mathcal{F}^{-1}\mathcal{F}[f](x, y).$$

Using definition 2.2 and 2.3 we get,

$$\mathcal{F}^{-1}[\mathcal{F}[f](\boldsymbol{\omega})](\boldsymbol{x}) = \mathcal{F}^{-1}[\hat{f}(\boldsymbol{\omega})](\boldsymbol{x}) = \frac{1}{2\pi} \int_{\mathbb{R}^2} \hat{f}(\boldsymbol{\omega}) e^{i\boldsymbol{\omega}\cdot\boldsymbol{x}} d\boldsymbol{\omega}.$$

In the integral, change $\boldsymbol{w} = (\omega_1, \omega_2)$ to 2D polar co-ordinates, (r, θ). With $\boldsymbol{\omega} = r\hat{\boldsymbol{r}} = (r\cos\theta, r\sin\theta)$. As this is a change of variables we require the determinant of the Jacobian,

$$|\mathcal{J}| = |\frac{\partial(\omega_1, \omega_2)}{\partial(r, \theta)}| = \begin{vmatrix} \frac{\partial \omega_1}{\partial r} & \frac{\partial \omega_1}{\partial \theta} \\ \frac{\partial \omega_2}{\partial r} & \frac{\partial \omega_2}{\partial \theta} \end{vmatrix} = \begin{vmatrix} \cos\theta & -r\sin\theta \\ \sin\theta & r\cos\theta \end{vmatrix} = r\cos^2\theta + r\sin^2\theta = r.$$

So the integral now becomes,

$$f(x, y) = \frac{1}{2\pi}\int_0^{2\pi}\int_0^{\infty} \hat{f}(r\hat{\boldsymbol{r}})e^{i(r\hat{\boldsymbol{r}})\cdot\boldsymbol{x}} r dr d\theta.$$

Now apply the Fourier Slice Theorem from Section 2.2.1, that $\sqrt{2\pi}\widehat{f}(r\hat{\boldsymbol{r}}) = \widehat{\mathcal{R}[f]}(r, \theta) = \hat{g}(r, \theta)$,

$$f(x, y) = \frac{1}{2\pi}\int_0^{2\pi}\int_0^{\infty} \big(\frac{1}{\sqrt{2\pi}}\hat{g}(r, \theta)\big)e^{i(r\hat{\boldsymbol{r}})\cdot\boldsymbol{x}} r dr d\theta$$

$$= \frac{1}{2\pi\sqrt{2\pi}}\int_0^{2\pi}\int_0^{\infty} \hat{g}(r, \theta)e^{i(r\hat{\boldsymbol{r}})\cdot\boldsymbol{x}} r dr d\theta.$$

This final equation we have stated is now in a form, where given $g(\theta, s)$, we can calculate $f(x, y)$. First we calculate the 1D Fourier Transform of $g(\theta, s)$, then sub into the equation,

$$f(x, y) = \frac{1}{2\pi\sqrt{2\pi}}\int_0^{2\pi}\int_0^{\infty} \hat{g}(r, \theta)e^{i(r\hat{\boldsymbol{r}})\cdot\boldsymbol{x}} r dr d\theta,$$

to get $f(x, y)$.

3 Numerical Algorithms

As was hinted at in the Introduction Section 1, we need to discretize this whole problem. One way to discretise the Radon transform, for a fixed angle θ_0 and a fixed perpendicular distance from origin s_0, is as follows,

$$g(\theta_0, s_0) = \int_{L_{(\theta_0, s_0)}} f(x(l), y(l)) dl,$$

$$\implies g(\theta_0, s_0) \approx \sum_{j=0}^{N_l} f(s_0\cos\theta_0 - l_j\sin\theta_0, s_0\sin\theta_0 + l_j\cos\theta_0)\Delta l,$$

where $\Delta l = \frac{l_{\max} - l_{\min}}{N_l}$, N_l denotes the number of subintervals along the ray (a finite segment of the line $L_{(\theta_0, s_0)}$ that passes through the image domain), resulting in $N_l + 1$ discrete nodes $l_j = l_0 + j\Delta l$. $l_{\min}(= l_0)$ and $l_{\max}$ are determined

by the size of the detector array/grid that encompasses the shape you wish to reconstruct, the image domain. We need to choose $l_{\min}$ and $l_{\max}$ such that the entire segment of the line $L(\theta_0, s_0)$, inside the image domain, is covered. Assume your image is supported in a rectangle, e.g. $x \in [x_{\min}, x_{\max}]$, $y \in [y_{\min}, y_{\max}]$. For fixed (θ_0, s_0) you first intersect the line,

$$x \cos\theta_0 + y \sin\theta_0 = s_0,$$

with the boundaries of the domain. For example sub in $y_{\max}$ to this equation to get x_1,

$$x_1 \cos\theta_0 + y_{\max} \sin\theta_0 = s_0.$$

If $x_1 < x_{\min}$ or $x_1 > x_{\max}$, then we disregard the point $(x_1, y_{\max})$ as it lies **outside** the image domain. But if $x_{\min} < x_1 < x_{\max}$, then this corresponds to the point $(x_1, y_{\max})$ that lies **inside** our image domain. We repeat this process for all $[x_{\min}, x_{\max}, y_{\min}(, y_{\max})]$, finding the two points that lie in the image domain. From these two points, say $(x_1, y_{\max})$ and $(x_{\max}, y_2)$, we compute the corresponding two l-values,

$$l = -x_1 \sin\theta_0 + y_{\max} \cos\theta_0.$$

Then out of those 2 values we calculated, $l \in \{l_1, l_2\}$, set,

$$l_{\min} = \min_{\{l_1, l_2\}} l, \qquad l_{\max} = \max_{\{l_1, l_2\}} l.$$

We have calculated the parameters of the discretisation, but how do we calculate $f(x(l_j), y(l_j))$. Well in reality, the discretised image is not represented as a continuous function f. The image f is projected onto a screen with a finite number of pixels. Each pixel has finite dimensions, typically taken to be a square of side length dx, i.e. of size $dx \times dx$.

For example, if our image f, was a black square, 1×1. And it was placed in the bottom-left corner of a white 5×5 discrete pixel image domain.

Let $p_{m,n}$ denote the pixel indexed by (m, n). Then f may be modelled as piecewise constant function,

$$f(x, y) = \begin{cases} 0, & (x, y) \in p_{0,0}, \\ 1, & (x, y) \notin p_{0,0}, \end{cases}$$

where 0 represents black, and 1 represents white. This motivates approximating the continuous image f as a piecewise constant function on a finite pixel grid, reflecting the discrete nature of digital images. That is, f is assumed to take a constant value $f_{m,n}$ on each pixel $p_{m,n}$, representing the image intensity within that pixel, such as the intensity values of the Shepp–Logan phantom shown in Figure 3. Under this approximation,

$$f(x(l_j), y(l_j)) \approx f_{m(j),n(j)} \text{ for any } (x(l_j), y(l_j)) \in p_{m(j),n(j)},$$

where for each sample point $(x(l_j), y(l_j))$, let $((m(j), n(j))$ denote the index of the pixel containing that point. So the integral becomes,

$$g(\theta_0, s_0) \approx \sum_{j=0}^{N_l} f_{m(j),n(j)} \Delta l.$$

If the pixel values $f_{m(j),n(j)}$ are stacked into a vector $f_k(j)$ using standard column-major choice, then for each sample point indexed by j, the pixel containing $(x(l_j), y(l_j))$ has indices $(m(j), n(j))$ and corresponding new linear index $k(j)$, such that,

$$f_{k(j)} = f_{m(j),n(j)}, \qquad k(j) = m(j) + Nn(j),$$

where N denotes the size of the grid, $N \times N$. And if we define a matrix A whose entries record the contribution of each ray to each pixel under the sampling discretisation described above. Specifically A_{pk} is obtained by counting the number of sample points $(x(l_j), y(l_j))$ of ray p that lie in pixel k, and multiplying this count by Δl.

This discretisation was for a fixed angle θ_0 and a fixed perpendicular distance from origin s_0. But we need to do this over a finite set of projection angles $\{\theta_i\}_{i=0}^{N_\theta}$ and perpendicular distances $\{s_j\}_{j=0}^{N_s}$. Hence,

$$g_{ij} = g(\theta_i, s_j).$$

Again stacking these sinogram data points, g_{ij}, into a vector using standard column-major choice we get,

$$g_p = g_{ij}, \qquad p = i + jN_\theta.$$

You can then put the approximate Radon Transform in matrix form,

$$\boldsymbol{g} = A\boldsymbol{f},$$

where $(\boldsymbol{f})_k = f_k$, $(A)_{pk} = A_{pk}$ and $(\boldsymbol{g})_p = g_p$. In most cases you can't just invert A, but you can find $\boldsymbol{f}$ that makes $A\boldsymbol{f}$ close to $\boldsymbol{g}$,

$$\min_{\boldsymbol{f}} \|A\boldsymbol{f} - \boldsymbol{g}\|_2. \tag{1}$$

If A is our Radon Transform $\mathcal{R}$, $\boldsymbol{f}$ is the reconstructed image, and $\boldsymbol{g}$ is our sinogram/data. Then finding $\boldsymbol{f}$, is our aim. And this sets up the optimisation problem that we want to solve.

3.1 Gradient Descent using Least Squares Algorithm

The gradient descent using least squares algorithm, is one of the many algorithms used to numerically solve the minimisation problem (1). But it instead solves the equivalent problem,

$$\min_{\boldsymbol{f}} \|A\boldsymbol{f} - \boldsymbol{g}\|_2^2,$$

which has the same minimisers as (1). But now the derivative is well defined

$$\nabla_{\boldsymbol{f}}\big(J(\boldsymbol{f})\big) = \frac{1}{2}\big(A^\top(A\boldsymbol{f}-\boldsymbol{g}) + (A\boldsymbol{f}-\boldsymbol{g})^T A\big) = A^\top(A\boldsymbol{f}-\boldsymbol{g}),$$

where $J(\boldsymbol{f}) = \frac{1}{2}\|A\boldsymbol{f}-\boldsymbol{g}\|^2 = \frac{1}{2}(A\boldsymbol{f}-\boldsymbol{g})^\top(A\boldsymbol{f}-\boldsymbol{g})$. Compared to,

$$\nabla_{\boldsymbol{f}}\,\|A\boldsymbol{f}-\boldsymbol{g}\|_2 = \frac{1}{\|A\boldsymbol{f}-\boldsymbol{g}\|_2}\,A^\top(A\boldsymbol{f}-\boldsymbol{g}).$$

Which is not defined at $A\boldsymbol{f}=\boldsymbol{g}$.

The numerical algorithm comes from the Gradient Descent Update Rule (GDUR). Which is a first order taylor expansion, similar to "truncating" the definition of the gradient. If we chooses a step size $h>0$ and some initial $\boldsymbol{f}^{(0)}$ value, GDUR says,

$$\begin{aligned}\boldsymbol{f}^{(k+1)} &= \boldsymbol{f}^{(k)} - h\nabla_{\boldsymbol{f}}\big(J(\boldsymbol{f}^{(k)})\big)\\ &= \boldsymbol{f}^{(k)} - hA^\top(A\boldsymbol{f}^{(k)}-\boldsymbol{g}).\end{aligned}$$

For this update rule to converge to $\boldsymbol{f}$, ie. $\boldsymbol{f}^{(k)}\to\boldsymbol{f}$ as $k\to\infty$, we must choose h such that $0<h<\frac{2}{\sigma^2_{\max}}$, where $\sigma^2_{\max}$ is the largest eigenvalue of $A^\top A$. This will be discussed in more detail in Section 3.2.

3.2 Adding Regularisation

The GDUR performs well when the measured data is noise-free, i.e. when $\boldsymbol{g}=\boldsymbol{g}_{\text{true}}$. However, when the measurements contain noise $\boldsymbol{\epsilon}\neq 0$, so that $\boldsymbol{g}=\boldsymbol{g}_{\text{true}}+\boldsymbol{\epsilon}$, the algorithm tends to amplify the noise during the iterations. As a result, the reconstructed solution becomes unstable. This motivates the introduction of regularisation, which we discuss below.

In Section 3.1, when looking at the problem $\min_{\boldsymbol{f}}\|A\boldsymbol{f}-\boldsymbol{g}\|_2^2$, the reader may have speculated that we can set the gradient equal to 0, and solve for $\boldsymbol{f}$. This is what we will start with in this Section.

$$\nabla_{\boldsymbol{f}}\big(J(\boldsymbol{f})\big) = 0,$$

$$\implies A^\top(A\boldsymbol{f}-\boldsymbol{g}) = 0 \implies \boldsymbol{f} = (A^\top A)^{-1}A^\top\boldsymbol{g}.$$

Now, let us decompose A using its Singular Value Decomposition (SVD),

$$A = U\Sigma V^\top,$$

$$A^\top = (U\Sigma V^\top)^\top = V\Sigma^\top U^\top,$$

where by construction, $U=[\,u_1,\dots,u_m\,], V=[\,v_1,\dots,v_n\,]$ are orthogonal matrices, and v_i,u_i are vectors. $\Sigma\in\mathbb{R}^{m\times n}$ is rectangular in general, since A is not necessarily square in CT, $\Sigma=\operatorname{diag}(\sigma_1,\dots,\sigma_r,0,\dots)$ with $\sigma_1\geq\cdots\geq\sigma_r>0$ and $(\Sigma)_{ii}=(\Sigma^\top)_{ii}=\sigma_i$. We have 0's in the diagonal of Σ as A may not be full rank, where $r=\operatorname{rank}(A)$.

$$\implies A^\top A = (V\Sigma^\top U^\top)(U\Sigma V^\top) = V\Sigma^\top\Sigma V^\top,$$

where $\Sigma^\top\Sigma = \text{diag}(\sigma_1^2, \ldots, \sigma_r^2, 0, \ldots, 0)$. And as U is orthogonal, $U^\top U = I$. So $\boldsymbol{f}$ becomes,

$$\boldsymbol{f} = (V\Sigma^\top\Sigma V^\top)^{-1}(V\Sigma^\top U^\top)\boldsymbol{g} = V(\Sigma^\top\Sigma)^{-1}\Sigma^\top U^\top \boldsymbol{g}.$$

where as V is also orthogonal, $V^{-1} = V^\top$. In general, the matrix $\Sigma^\top\Sigma$ is not invertible, since Σ may not be full-rank. Consequently, $(\Sigma^\top\Sigma)^{-1}\Sigma^\top$ will not be well-defined. However, to understand the instability of the unregularised least-squares solution, we temporarily consider the idealised case in which Σ is full rank. In this case, $\Sigma^\top\Sigma$ is invertible and $(\Sigma^\top\Sigma)^{-1}\Sigma^\top$ exists. If we then set $\boldsymbol{c} = (\Sigma^\top\Sigma)^{-1}\Sigma^\top U^\top \boldsymbol{g} \implies \boldsymbol{f} = V\boldsymbol{c}$. The components of $\boldsymbol{c}$ will become $c_i = \frac{u_i^\top}{\sigma_i}\boldsymbol{g}$, so the SVD expansion becomes,

$$\boldsymbol{f} = \sum_{i=1}^{r} \frac{\langle u_i, \boldsymbol{g}\rangle}{\sigma_i} v_i.$$

As you can see if σ_i is small then any nonzero contribution $\langle u_i, \boldsymbol{g}\rangle$ is amplified by the factor $\frac{1}{\sigma_i}$. In the presence of noise, such contributions are unavoidable, leading to instability in the reconstructed solution. As above, writing $\boldsymbol{g} = \boldsymbol{g}_{\text{true}} + \boldsymbol{\epsilon}$, the coefficients of the SVD expansion satisfy

$$\frac{\langle u_i, \boldsymbol{g}\rangle}{\sigma_i} = \frac{\langle u_i, \boldsymbol{g}_{\text{true}}\rangle}{\sigma_i} + \frac{\langle u_i, \boldsymbol{\epsilon}\rangle}{\sigma_i}.$$

Since the noise term $\boldsymbol{\epsilon} \neq 0$, the contribution $\frac{\langle u_i, \boldsymbol{\epsilon}\rangle}{\sigma_i}$ becomes large for small σ_i, causing the solution for $\boldsymbol{f}$ to blow up. We can see how this affects the GDUR, by subbing in $\boldsymbol{f}^{(k)} = V\boldsymbol{c}^{(k)}$, $A = U\Sigma V^\top$ and $A^\top = V\Sigma^\top U^\top$,

$$\begin{aligned}
\implies V\boldsymbol{c}^{(k+1)} &= V\boldsymbol{c}^{(k)} - h(V\Sigma^\top U^\top)((U\Sigma V^\top)V\boldsymbol{c}^{(k)} - \boldsymbol{g}) \\
&= V\boldsymbol{c}^{(k)} - h(V\Sigma^\top\Sigma\boldsymbol{c}^{(k)} - V\Sigma^\top U^\top\boldsymbol{g}) \\
&= V(\boldsymbol{c}^{(k)} - h(\Sigma^\top\Sigma\boldsymbol{c}^{(k)} - \Sigma^\top U^\top\boldsymbol{g})).
\end{aligned}$$

Left multiply by V^{-1} on both sides,

$$\begin{aligned}
\implies \boldsymbol{c}^{(k+1)} &= \boldsymbol{c}^{(k)} - h(\Sigma^\top\Sigma\boldsymbol{c}^{(k)} - \Sigma^\top U^\top\boldsymbol{g}), \\
\boldsymbol{c}^{(k+1)} &= (I - h\Sigma^\top\Sigma)\,\boldsymbol{c}^{(k)} + h\Sigma^\top U^\top\boldsymbol{g}.
\end{aligned}$$

Writing in component form,

$$c_i^{(k+1)} = (1 - h\sigma_i^2)\,c_i^{(k)} + h\sigma_i(u_i^\top\boldsymbol{g}).$$

We notice that, this is a linear scalar recurrence. Such a recurrence converges for $k \to \infty$ if and only if

$$|1 - h\sigma_i^2| < 1 \quad \text{for all } i.$$

See Appendix A.1 for proof of this inequality. Solving this inequality gives

$$-1 < 1 - h\sigma_i^2 < 1 \quad \iff \quad 0 < h\sigma_i^2 < 2 \quad \iff \quad 0 < h < \frac{2}{\sigma_i^2}.$$

Since this condition must hold for every singular value, the most restrictive constraint comes from the largest singular value $\sigma_{\max} = \max_i \sigma_i$, hence a sufficient and necessary condition for convergence is

$$0 < h < \frac{2}{\sigma_{\max}^2}.$$

This is how you choose step size, as stated in Section 3.1. Also if $\sigma_i << 1 \implies 1 - h\sigma_i^2 \approx 1$, then

$$c_i^{(k+1)} \approx c_i^{(k)} + h\sigma_i \langle u_i, \boldsymbol{g} \rangle.$$

So $c_i^{(k+1)} - c_i^{(k)} \approx h\sigma_i \langle u_i^\top, \boldsymbol{g} \rangle << 1$, this means GDUR will have small gaps in values for it's iterations, hence will take a long time to converge to $\boldsymbol{f}$.

To fix this problem we add regularisation, $\alpha \|\boldsymbol{f}\|_2^2$. This term penalises small values of σ_i, by adding a larger term α. The minimisation problem with regularisation becomes,

$$\min_{\boldsymbol{f}} (\|A\boldsymbol{f} - \boldsymbol{g}\|_2^2 + \alpha \|\boldsymbol{f}\|_2^2).$$

And following the same steps as before, calculate the gradient and set to 0,

$$\nabla_{\boldsymbol{f}} \big(\frac{1}{2} (\|A\boldsymbol{f} - \boldsymbol{g}\|_2^2 + \alpha \|\boldsymbol{f}\|_2^2) \big) = A^\top (A\boldsymbol{f} - \boldsymbol{g}) + \alpha \boldsymbol{f} = 0,$$

$$\implies (A^\top A + \alpha I)\boldsymbol{f} = A^\top \boldsymbol{g}.$$

Sub-in the same SVD expressions as calculated earlier for A,

$$(V\Sigma^\top \Sigma V^\top + \alpha I)\boldsymbol{f} = V\Sigma^\top U^\top \boldsymbol{g},$$

$$\implies V(\Sigma^\top \Sigma + \alpha I)V^\top \boldsymbol{f} = V\Sigma^\top U^\top \boldsymbol{g}.$$

Left multiply by V^{-1},

$$(\Sigma^\top \Sigma + \alpha I)V^\top \boldsymbol{f} = \Sigma^\top U^\top \boldsymbol{g}.$$

Similar to before, set $\boldsymbol{c} = V^\top \boldsymbol{f}$,

$$(\Sigma^\top \Sigma + \alpha I)\boldsymbol{c} = \Sigma^\top U^\top \boldsymbol{g}.$$

Writing in component form,

$$(\sigma_i^2 + \alpha)c_i = \sigma_i (u_i^\top \boldsymbol{g}),$$

$$\implies c_i = \frac{\sigma_i}{\sigma_i^2 + \alpha} \langle u_i, \boldsymbol{g} \rangle.$$

And therefore the SVD expansion is,

$$\boldsymbol{f} = V\boldsymbol{c} = \sum_{i=1}^{r} \frac{\sigma_i}{\sigma_i^2 + \alpha} \langle u_i, \boldsymbol{g} \rangle v_i.$$

Like last time, if $\sigma_i << 1$ and $\sigma_i << \alpha$, but this time the solution doesn't blow up,

$$\frac{\sigma_i}{\sigma_i^2 + \alpha} \approx \frac{\sigma_i}{\alpha}.$$

This means the solution is stabilised.

3.3 Proximal Maps

This is all well and good when the objective function $\boldsymbol{f}$ is smooth. However, in many inverse problems the regularisation term is not differentiable, meaning that the gradient is not well defined everywhere and standard gradient descent cannot be applied directly. To deal with this, we rewrite the optimisation problem by splitting the objective into a smooth and a (possibly) non-smooth part. From this point on, we use the generic variable x to represent the optimisation variable, rather than the image f, to match standard optimisation notation. Let x denote the variable we wish to optimise, instead of $\boldsymbol{f}$. We define

$$f(x) = \frac{1}{2}\|Ax - b\|_2^2, \qquad g(x) = \alpha R(x),$$

where f is smooth and R may be non-smooth. The optimisation problem can then be written as

$$\min_{x \in \mathbb{R}^n} \; f(x) + g(x).$$

This formulation allows us to apply proximal gradient descent, in which the smooth term is handled via its gradient and the non-smooth term via its proximal map. The following definitions are from Convex Analysis [4].

Definition 3.1 (Proximal Map) *Let $g : \mathbb{R}^n \to (-\infty, +\infty]$ be convex, non-differentiable function. The proximal map of g with parameter $\lambda > 0$ is defined by,*

$$\text{prox}_{\lambda g}(y) = \arg\min_{x \in \mathbb{R}^n} \left\{ g(x) + \frac{1}{2\lambda}\|x - y\|^2 \right\}.$$

Definition 3.2 (Convex function) *A function $f : \mathbb{R}^n \to (-\infty, +\infty]$ is called* convex *if for all $x, y \in \mathbb{R}^n$ and all $\lambda \in [0, 1]$,*

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda) f(y).$$

As an example of the use of the definition of the proximal map 3.1, take the function $g(x) = |x|$ and evaluate it's proximal map,

$$\text{prox}_{\lambda g}(y) = \arg\min_{x \in \mathbb{R}^n} \left\{ |x| + \frac{1}{2\lambda}\|x - y\|^2 \right\} = \arg\min_{x \in \mathbb{R}^n} \{h(x)\}.$$

For $x > 0$,

$$h'(x) = 1 + \frac{1}{\lambda}(x - y) = 0 \implies x = -\lambda + y.$$

For $x < 0$,

$$h'(x) = -1 + \frac{1}{\lambda}(x - y) = 0 \implies x = \lambda + y.$$

$$\implies \operatorname{prox}_{\lambda g}(y) = \begin{cases} y - \lambda, & y > \lambda, \\ y + \lambda, & y < -\lambda, \\ 0, & |y| \leq \lambda. \end{cases}$$

This shows that large $\pm$ values of y get shrunken towards 0 by λ and small values, $y \in [-\lambda, \lambda]$ are killed off. The proximal map should not be interpreted as a way of differentiating non-smooth functions. Instead, it provides an alternative update operator that replaces the GDUR for the non-smooth term, yielding a well-defined algorithm. In the case $g(x) = |x|$, although the gradient is undefined at $x = 0$, the proximal map is single-valued everywhere. And hence, using the definition 3.1 we can reformulate the GDUR as the Proximal Gradient Descent Update Rule (PGDUR), with step size $\lambda > 0$,

$$x^{(k+1)} = \operatorname{prox}_{\lambda g}\left(x^{(k)} - \lambda \nabla(f(x^{(k)}))\right).$$

This is sometimes known as the Iterative Soft Thresholding Algorithm (ISTA) in literature. To show the relationship between the PGDUR and GDUR, we consider the case when $g(x)$ is the Tikhonov regularisation,

$$g(x) = \frac{\alpha}{2}\|x\|_2^2, \qquad \alpha > 0.$$

And compute the proximal map,

$$\operatorname{prox}_{\lambda g}(v) = \arg\min_{x \in \mathbb{R}^n} \left\{ \frac{\alpha}{2}\|x\|^2 + \frac{1}{2\lambda}\|x - v\|^2 \right\}.$$

Differentiating the expression in the curly brackets w.r.t x and set the gradient to zero,

$$\alpha x + \frac{1}{\lambda}(x - v) = 0.$$

Re-arrange and collecting terms, we get,

$$\left(\alpha + \frac{1}{\lambda}\right) x = \frac{1}{\lambda} v \implies x = \frac{1}{1 + \lambda\alpha}\, v.$$

Therefore the argument that minimises the expression in the curly brackets, which is defined to be proximal map is,

$$\operatorname{prox}_{\lambda g}(v) = \frac{1}{1 + \lambda\alpha}\, v.$$

Using this expression in terms of v, the PGDUR becomes, where $v = x^{(k)} - \lambda\nabla\big(f(x^{(k)})\big)$ and $f(x^{(k)}) = \frac{1}{2}\|Ax^{(k)} - b\|_2^2$,

$$x^{(k+1)} = \frac{1}{1+\lambda\alpha}\left(x^{(k)} - \lambda(A^\top(Ax^{(k)} - b))\right),$$

$$x^{(k+1)} = \frac{1}{1+\lambda\alpha}\Big((1+\lambda\alpha)x^{(k)} - \lambda\alpha x^{(k)} - \lambda A^\top\big(Ax^{(k)} - b\Big),$$

$$x^{(k+1)} = x^{(k)} - \frac{\lambda}{1+\lambda\alpha}\big(\alpha x^{(k)} + A^\top(Ax^{(k)} - b)\big).$$

But notice as,

$$\alpha x + A^\top(Ax - b) = \nabla\left(\frac{1}{2}\|Ax - b\|^2 + \frac{\alpha}{2}\|x\|^2\right) = \nabla\big(J(x)\big).$$

Thus the update rule recovers,

$$x^{(k+1)} = x^{(k)} - h\big(\nabla J(x^{(k)})\big), \qquad h = \frac{\lambda}{1+\lambda\alpha}.$$

The expression above is simply the GDUR with step size $h = \frac{\lambda}{1+\lambda\alpha} > 0$.

3.4 Primal & Dual Problem

Generalising the new reformulated problem from Section 3.3 to allow for a general operator $K : \mathbb{R}^n \to \mathbb{R}^m$, to be applied on the regulariser we get the primal problem,

$$\min_{x\in\mathbb{R}^n}\big(f(x) + g(Kx)\big). \tag{2}$$

Introducing a new variable $z = Kx$, this problem can be equivalently written, as the constrained optimisation problem. Which in convex optimisation [5], the dual problem is defined via the Lagrangian of the constrained primal problem,

$$\min_{x\in\mathbb{R}^n, z\in\mathbb{R}^m}\big(f(x) + g(z)\big), \qquad \text{s.t. } z - Kx = 0,$$

where $f(x) + g(z)$ is the primal objective function. Introduce Lagrange multipliers $\mu \in \mathbb{R}^m$ for the equality constraint. The Lagrangian is defined as

$$\mathcal{L}(x, z, \mu) = f(x) + g(z) + \mu^\top(Kx - z).$$

The dual function is obtained by minimising the Lagrangian over all primal variables x, z,

$$q(\mu) = \inf_{x,z}\mathcal{L}(x, z, \mu).$$

A key property of the dual function [5], is that,

$$q(\mu) \le f(x) + g(z) \qquad \text{for all feasible } (x, z) \text{ satisfying } z = Kx,$$

so $q(\mu)$ provides a lower bound on the primal objective function. The dual problem is defined as

$$\max_{\mu} q(\mu).$$

That is, the dual problem maximises the best lower bound on the primal objective function obtained from the Lagrangian. As an example consider the problem of minimizing $x^\top x = \|x\|_2^2$, subject to $Kx = z$. The Lagrangian of this problem is,

$$\mathcal{L}(x, \mu) = x^\top x + \mu^\top (Kx - z) = x^\top x + \langle \mu, Kx - z \rangle.$$

Therefore the dual function of this problem is given by $q(\mu) = \inf_x \mathcal{L}(x, z, \mu)$. To work this out for the Lagrangian, we compute $\nabla_x \mathcal{L} = 0$,

$$\nabla_x \mathcal{L}(x, \mu) = 2x + K^\top \mu = 0,$$

$$\implies x = -\frac{1}{2} K^\top \mu.$$

Therefore the dual Function is,

$$q(\mu) = \mathcal{L}(-\frac{1}{2}K^\top \mu, \mu) = (-\frac{1}{2}K^\top \mu)^\top (-\frac{1}{2}K^\top \mu) + \mu^\top (K(-\frac{1}{2}K^\top \mu) - z)$$

$$= -\frac{1}{4}(\mu^\top K K^\top \mu) - \mu^\top z.$$

So the dual property states that,

$$-\frac{1}{4}(\mu^\top K K^\top \mu) - \mu^\top z \leq \inf\{x^\top x | Kx = z\}.$$

To formulate a general form of the dual problem, we will need the definition of the convex conjugate of a function. The following definitions are from Convex Analysis [4].

Definition 3.3 (Convex Conjugate) *Let $f : \mathbb{R}^n \to (-\infty, +\infty]$ be a proper function. The convex conjugate (or Fenchel conjugate) of f is the function*

$$f^*(y) = \sup_{x \in \mathbb{R}^n} \big(\langle y, x \rangle - f(x)\big).$$

Definition 3.4 (Proper function) *A function $f : \mathbb{R}^n \to (-\infty, \infty]$ is called proper if:*

1. *$f(x) > -\infty$ for all $x \in \mathbb{R}^n$, and*
2. *there exists at least one $x_0 \in \mathbb{R}^n$ such that $f(x_0) < \infty$.*

Consider the primal problem

$$\min_{x \in \mathbb{R}^n} \big(f(x) + g(Kx)\big),$$

where $f : \mathbb{R}^n \to (-\infty, +\infty]$ and $g : \mathbb{R}^m \to (-\infty, +\infty]$ are now both proper convex functions, and $K : \mathbb{R}^n \to \mathbb{R}^m$ is linear. The Dual problem is formulated as,

$$\max_{y \in \mathbb{R}^m} \; -f^*(-K^\top y) - g^*(y),$$

see derivation of this formula in Appendix A.2. To help in understanding this dual problem formula, consider the same problem as before. Minimizing $x^\top x$, subject to $Kx = z$. First write this in the primal problem form,

$$f(x) = x^\top x = |x|^2,$$

$$g(y = Kx) = \begin{cases} 0, & \text{if } y = z, \\ \infty, & \text{otherwise,} \end{cases}$$

where $g(y)$ is an indicator function, and handles the constraints of the problem. Using the definition for convex conjugate 3.3,

$$f^*(y) = \sup_{x \in \mathbb{R}^n} \big(\langle y, x \rangle - |x|^2 \big) = \sup_{x \in \mathbb{R}^n} \big(J(x) \big).$$

To find out where the supremum is, calculate the gradient and set it equal to 0, and solve for x,

$$\nabla_x J(x) = y - 2x = 0,$$

$$\implies x = \frac{y}{2}.$$

Sub this expression for x back into $f^*(y)$ to get,

$$f^*(y) = \langle y, \frac{y}{2} \rangle - |\frac{y}{2}|^2 = \frac{|y|^2}{2} - \frac{|y|^2}{4} = \frac{|y|^2}{4}.$$

Next, calculate

$$g^*(y) = \sup_{x \in \mathbb{R}^n} \big(\langle y, x \rangle - g(x) \big).$$

Clearly to make $(\langle y, x \rangle - g(x))$ as big as possible, we need to choose $g(x) = 0$ not $g(x) = \infty$. This only happens at $x = z$, due to how we defined our indicator function above. Where we've just used a different input variable, x, instead of y. Therefore,

$$g^*(y) = \langle y, z \rangle.$$

Bringing this all together we can explicitly define the dual problem for this problem,

$$\max_{y \in \mathbb{R}^m} \; -f^*(-K^\top y) - g^*(y) = \max_{y \in \mathbb{R}^m} \; -\frac{|K^\top y|^2}{4} - \langle y, z \rangle.$$

If $y = \mu$, we recover the same dual property inequality as above.

3.5 PDHG Algorithm

The PDHG algorithm is designed to solve the generic saddle-point problem of the form:

$$\min_{x\in\mathbb{R}^n}\max_{y\in\mathbb{R}^m}\langle Kx,y\rangle + f(x) - g^*(y), \tag{3}$$

where $f:\mathbb{R}^n\to(-\infty,+\infty]$, $g:\mathbb{R}^m\to(-\infty,+\infty]$ are proper, convex, and lower semicontinuous functions, and $K:\mathbb{R}^n\to\mathbb{R}^m$ is a linear operator. The following definition is from Convex Analysis [4].

Definition 3.5 (Lower semicontinuous function) *A function $f:\mathbb{R}^n\to(-\infty,+\infty]$ is called lower semicontinuous if for every $x\in\mathbb{R}^n$ and every sequence (x_k) with $x_k\to x$, we have*

$$f(x)\le\liminf_{k\to\infty} f(x_k).$$

The generic saddle-point problem can be decomposed into the primal problem and it's corresponding dual problem from Section 3.4. And the update rules for the PDHG algorithm is:-

- Gradient ascent for the dual problem:

$$\implies y^{(k+1)} = \operatorname{prox}_{\tau g^*}\left(y^{(k)} + \tau K\bar{x}^{(k)}\right).$$

- Gradient descent for the primal problem:

$$\implies x^{(k+1)} = \operatorname{prox}_{\lambda f}\left(x^{(k)} - \lambda K^* y^{(k+1)}\right).$$

- Over-relaxation of the primal variable:

$$\implies \bar{x}^{(k+1)} = x^{(k+1)} + \theta\left(x^{(k+1)} - x^{(k)}\right).$$

We now show that the gradient descent update rule for the primal problem comes from fixing $y = y^{(k+1)}$, so that the generic saddle point problem 3 reduces to the primal problem 2 from Section 3.4,

$$\min_{x\in\mathbb{R}^n}\langle Kx, y^{(k+1)}\rangle + f(x),$$

where we have ignored maximising y, as it is already fixed. Compare this to the PGDUR we stated in Section 3.3,

$$x^{(k+1)} = \operatorname{prox}_{\lambda f}\big(x^{(k)} - \lambda\nabla_x g(x^{(k)})\big),$$

where we have swapped the function names. If we choose $g(x^{(k)}) = \langle Kx^{(k)}, y\rangle = \langle x^{(k)}, K^*y^{(k+1)}\rangle$, the PGDUR becomes,

$$\implies \operatorname{prox}_{\lambda f}\big(x^{(k)} - \lambda\nabla_x\langle x^{(k)}, K^*y^{(k+1)}\rangle\big) = \operatorname{prox}_{\lambda f}\big(x^{(k)} - \lambda K^*y^{(k+1)}\big).$$

We have recovered the update rule for the primal problem of the generic saddle-point problem. Now to show the gradient ascent update rule for the dual problem. First fix $x = \bar{x}^{(k)}$, so the generic saddle point problem 3 becomes

$$\max_{y \in \mathbb{R}^m} \langle K\bar{x}^{(k)}, y \rangle - g^*(y),$$

where we have ignored minimizing x, as it is constant. Using the same method as before, but this time we choose $g(y^{(k)}) = \langle K\bar{x}^{(k)}, y^{(k)} \rangle$, the PGAUR becomes

$$y^{(k+1)} = \text{prox}_{\tau g^*}\left(y^{(k)} + \tau \nabla_y \langle K\bar{x}^{(k)}, y^{(k)} \rangle\right) = \text{prox}_{\tau g^*}\left(y^{(k)} + \tau K\bar{x}^{(k)}\right).$$

We have recovered the update rule for the dual problem of the generic saddle-point problem. The reasoning behind the over-relaxation of the primal variable is omitted for this project, see the paper by Chambolle and Pock for more details [6]. The paper also includes details on appropriate step size conditions required for convergence.

As a final example, let's look at a non-differentiable function, the Anisotropic Discrete Total Variation (TV) norm. Let x denote a discrete image defined on an $N \times N$ pixel grid, so that $x \in \mathbb{R}^{N \times N}$, with entries x_{ij} representing the pixel intensity at grid location (i, j). Similar to how we described the discretisation in Section 3. Under this identification, the Anisotropic Discrete TV norm is defined as,

$$g(Kx) = \|x\|_{TV} = \sum_{ij} |x_{ij} - x_{i(j+1)}| + |x_{ij} - x_{(i+1)j}|.$$

Now calculate the Gradient ascent for the dual problem using this norm. Define K to be the discrete gradient that works as follows,

$$(Kx)_{ijk} = \begin{cases} x_{ij} - x_{i(j+1)}, & k = 1, \\ x_{ij} - x_{(i+1)j}, & k = 2. \end{cases}$$

We now re-write $g(Kx = y)$ as,

$$g(y) = \sum_{ijk} |y_{ijk}|.$$

The Gradient ascent update rule for the dual problem requires calculating the proximal map of $g^*(y)$,

$$\text{prox}_{\tau g^*}(v) = \arg\min_{y \in \mathbb{R}^m} \left\{ g^*(y) + \frac{1}{2\tau} \|y - v\|^2 \right\}.$$

Let's first calculate the convex conjugate function $g^*(y)$, using Definition 3.3.

$$g^*(y) = \sup_x \left\{ \langle y, x \rangle - g(x) \right\},$$

$$\implies \sup_x \left\{ \sum_{ijk} y_{ijk} x_{ijk} - \sum_{ijk} |x_{ijk}| \right\},$$

$$\implies \sum_{ijk} \sup_x \left\{ y_{ijk} x_{ijk} - |x_{ijk}| \right\}.$$

Where we have interchanged the supremum and the sum because, the optimisation variable $x = (x_{ijk})$ is unconstrained and the objective function is separable across the indices (i, j, k), the supremum can be computed componentwise. Each resulting maximisation problem involves a single scalar variable and is identical for every choice of indices. Hence we can drop the indices, meaning we're only considering the problem,

$$\sup_x \left\{ xy - |x| \right\}.$$

Considering cases, $x > 0$,

$$\implies xy - x = x(y-1),$$

$$\implies \sup_{x>0} \left\{ xy - |x| \right\} = \begin{cases} \infty, & y > 1, \\ 0, & y \leq 1. \end{cases}$$

The case $x < 0$,

$$\implies xy + x = x(y+1),$$

$$\implies \sup_{x<0} \left\{ xy - |x| \right\} = \begin{cases} \infty, & y < -1, \\ 0, & y \geq -1. \end{cases}$$

Combining these two cases, we get,

$$\sup_x \left\{ xy - |x| \right\} = \begin{cases} \infty, & y < -1 \text{ or } y > 1, \\ 0, & -1 \leq y \leq 1. \end{cases}$$

Restoring the indices, this yields

$$g^*(y) = \sum_{i,j,k} \begin{cases} \infty, & |y_{ijk}| > 1, \\ 0, & |y_{ijk}| \leq 1. \end{cases}$$

Hence $g^*(y)$ is finite if and only if $|y_{ijk}| \leq 1$ for all indices (i, j, k). Therefore, minimising over all $y \in \mathbb{R}^m$ is equivalent to minimising over the restriction that $|y_{ijk}| \leq 1$ for all indices (i, j, k). So this means the proximal map becomes,

$$\text{prox}_{\tau g^*}(v) = \arg \min_{|y_{ijk}| \leq 1} \left\{ \frac{1}{2\tau} \|y - v\|^2 \right\} \quad \text{for all (i,j,k).}$$

Expanding the squared norm gives

$$\|y - v\|^2 = \sum_{i,j,k} (y_{ijk} - v_{ijk})^2.$$

Since both the objective function and the constraint $-1 \leq y_{ijk} \leq 1$ act independently on each component, the minimisation can be performed separately for each index (i, j, k). For a fixed component, we therefore consider the scalar problem

$$\min_{-1 \leq y \leq 1} (y - v_{ijk})^2.$$

This has the explicit solution

$$y_{ijk} = \begin{cases} 1, & v_{ijk} > 1, \\ -1, & v_{ijk} < -1, \\ v_{ijk}, & -1 \leq v_{ijk} \leq 1. \end{cases}$$

Applying this rule to every component yields the proximal operator $\text{prox}_{\tau g^*}(v)$. And substituting $v = y^{(k)} + \tau K \bar{x}^{(k)}$ into the PDHG dual update rule, we get,

$$y_{ijk}^{(k+1)} = \begin{cases} 1, & y_{ijk}^{(k)} + \tau (K \bar{x}^{(k)})_{ijk} > 1, \\ -1, & y_{ijk}^{(k)} + \tau (K \bar{x}^{(k)})_{ijk} < -1, \\ y_{ijk}^{(k)} + \tau (K \bar{x}^{(k)})_{ijk}, & -1 \leq y_{ijk}^{(k)} + \tau (K \bar{x}^{(k)})_{ijk} \leq 1. \end{cases}$$

As you can see, this form is calculable. It has no non-differentiable points.

4 Conclusion

This project aimed to provide the reader with a clear path from fundamental analytical methods all the way to modern numerical algorithms used in image reconstruction. Starting with the result of Radon's paper [2] and leading up to the Filtered Back Projection Method, we formulated an analytical method to reconstruct the image. Following from that, we discretised our analytical methods, in particular the Radon Transform, due to the finite and discrete nature of image reconstruction in real life applications. Due to not all solutions to the discretised matrix form of the Inverse Radon Transform being defined, we had to come up with numerical methods to solve the problem. We then developed these numerical methods by adding regularisation, proximal maps and separating the problem into the primal and dual problem. These extensions to the numerical methods helped stabilise the solution and allowed for non-differentiable functions.

Given more time for the project, I would have looked at the PDHG algorithm in more detail, including its convergence and the purpose of the over-relaxation of the primal variable. I would have included a Section on the Core Imaging Library (CIL) package [7]. Which is an open-source, mainly Python framework, for reconstructing and visualising tomographic data. CIL provides optimised standard methods such as Filtered Back Projection and extensive optimisation framework for prototyping reconstruction methods including total variation regularisation.

A Appendix

A.1 Convergence Proof

Proof that the recurrence relation,

$$c_i^{(k+1)} = (1 - h\sigma_i^2)\, c_i^{(k)} + h\sigma_i \langle u_i, \boldsymbol{g} \rangle.$$

Only converges for $k \to \infty$ if and only if

$$|1 - h\sigma_i^2| < 1 \quad \text{for all } i.$$

First re-write the recurrence relation in the form

$$x_{k+1} = ax_k + b,$$

where $x_k = c_i^{(k)}$, $a = 1 - h\sigma_i^2$, $b = h\sigma_i \langle u_i, \boldsymbol{g} \rangle$. If we set $k + 1 = n$, we get,

$$x_n = ax_{n-1} + b.$$

Now substitute in recursively,

$$x_n = a(ax_{n-2} + b) + b = a^2 x_{n-2} + ab + b$$

$$= a^2(ax_{n-3} + b) + ab + b = a^3 x_{n-3} + a^2 b + ab + b$$

$$\vdots$$

$$= a^n x_{n-n} + \sum_{n=0}^{n-1} a^n b = a^n x_0 + \frac{1 - a^n}{1 - a} b,$$

where the fraction in the last line comes from the formula for the sum to n of a geometric series. As $n \to \infty$, if $|a| < 1 \implies a^n \to 0$ and $x_n \to \frac{b}{1-a}$, but if $|a| \geq 1$ then a^n does not decay, it instead diverges. So convergence requires,

$$|a| = |1 - h\sigma_i^2| < 1.$$

A.2 Dual Problem Formula Derivation

Consider the primal problem from Section 3.4

$$\min_{x \in \mathbb{R}^n} \big(f(x) + g(Kx) \big),$$

where $f : \mathbb{R}^n \to (-\infty, +\infty]$, $g : \mathbb{R}^m \to (-\infty, +\infty]$ are proper, convex, and lower semi-continuous functions, and $K : \mathbb{R}^n \to \mathbb{R}^m$ is a linear operator. For any convex function g, its convex conjugate g^* is defined, as in Definition 3.3, by

$$g^*(y) = \sup_{z \in \mathbb{R}^n} \{ \langle y, z \rangle - g(z) \}.$$

Equivalently, as g is proper, convex, and lower semicontinuous, then g admits the representation,

$$g(z) = \sup_{y \in \mathbb{R}^m} \{\langle y, z\rangle - g^*(y)\}.$$

This result can be seen in this book on Convex Analysis [4]. Applying this with $z = Kx$, we obtain $g(Kx) = \sup_{y\in\mathbb{R}^m} \{\langle y, Kx\rangle - g^*(y)\}$. Substituting this expression into the Primal problem gives,

$$\min_{x\in\mathbb{R}^n} \big(f(x) + g(Kx)\big) = \min_{x\in\mathbb{R}^n} \big(f(x) + \sup_{y\in\mathbb{R}^m} \{\langle y, Kx\rangle - g^*(y)\}\big)$$

$$= \min_{x\in\mathbb{R}^n} \sup_{y\in\mathbb{R}^m} \{f(x) + \langle y, Kx\rangle - g^*(y)\} = \min_{x\in\mathbb{R}^n} \sup_{y\in\mathbb{R}^m} W(x, y).$$

This expression has the saddle-point structure used by the PDHG algorithm, see Section 3.5. As f, g are proper, convex, and lower semi-continuous functions. By Sion's Minimax theorem [8], we may interchange the order of minimisation and supremum,

$$\min_{x\in\mathbb{R}^n} \sup_{y\in\mathbb{R}^m} W(x, y) = \sup_{y\in\mathbb{R}^m} \min_{x\in\mathbb{R}^n} W(x, y).$$

Thus,

$$\min_{x\in\mathbb{R}^n} \sup_{y\in\mathbb{R}^m} W(x, y) = \sup_{y\in\mathbb{R}^m} \left\{-g^*(y) + \min_{x\in\mathbb{R}^n} \big(f(x) + \langle K^\top y, x\rangle\big)\right\},$$

where we used $\langle y, Kx\rangle = \langle K^\top y, x\rangle$. Consider the inner minimisation problem

$$\min_{x\in\mathbb{R}^n} \big(f(x) + \langle K^\top y, x\rangle\big) = -\sup_{x\in\mathbb{R}^n} \big\{\langle -K^\top y, x\rangle - f(x)\big\}$$

$$= -f^*(-K^\top y),$$

where we have used $\inf A = -\sup(-A)$, see proof of this in Appendix A.3, and the definition of convex conjugate of $-K^\top y$. Substituting back, we obtain the dual problem

$$\sup_{y\in\mathbb{R}^m} \big\{-g^*(y) - f^*(-K^\top y)\big\} = \max_{y\in\mathbb{R}^m} -f^*(-K^\top y) - g^*(y).$$

A.3 Relation Between infA and supA Proof

Prove

$$\inf A = -\sup(-A).$$

Where $A \subset \mathbb{R}$, and $a \in A$. Hence the natural definition of the reflected set is, $-A = \{-a : a \in A\}$. Before we start the proof, let's define the supremum and infimum.

Definition A.1 (Supremum) *For $s = \sup A$, the least upper bound is defined as,*

1. $a \leq s$ *for all* $a \in A$ *(so* s *is an upper bound of* A*).*

2. *for every* $\varepsilon > 0$*, there exists* $a \in A$ *such that*

$$s - \varepsilon < a \leq s.$$

Definition A.2 (Infimum) *For* $t = \inf A$*, the greatest lower bound is defined as,*

1. $t \leq a$ *for all* $a \in A$ *(so* t *is a lower bound of* A*).*

2. *for every* $\varepsilon > 0$*, there exists* $a \in A$ *such that*

$$t \leq a < t + \varepsilon.$$

Let $m = \inf A$. By definition A.2,

$$m \leq a \quad \text{for all } a \in A.$$

$$\implies -m \geq -a \quad \text{for all } a \in A.$$

Hence, $-m$ is an upper bound of the set $-A$, not necessarily the least upper bound. Consequently,

$$\sup(-A) \leq -m.$$

Now check that there exists no smaller upper bound than $-m$. Let $u < -m$, be an upper bound. Then $-u > m$. Since $m = \inf A$, the number $-u$ cannot be a lower bound of A, by definition of infimum. Therefore, there exists some $a \in A$ such that

$$(m <)a < -u \implies -a > u.$$

Thus, u fails to be an upper bound of $-A$. And by contradiction it follows that $-m$ is the least upper bound of $-A$, and hence

$$\sup(-A) = -m \implies \sup(-A) = -\inf A.$$

References

[1] Johann Radon. "Über die Bestimmung von Funktionen durch ihre Integralwerte längs gewisser Mannigfaltigkeiten". In: *Berichte über die Verhandlungen der Königlich-Sächsischen Akademie der Wissenschaften zu Leipzig, Mathematisch-Physische Klasse* 69 (1917), pp. 262–277.

[2] Johann Radon. "On the determination of functions from their integral values along certain manifolds". In: *IEEE Transactions on Medical Imaging* 5.4 (1986), pp. 170–176. DOI: 10.1109/TMI.1986.4307775.

[3] Per Christian Hansen, Jakob Jørgensen, and William R. B. Lionheart. *Computed Tomography: Algorithms, Insight, and Just Enough Theory*. Ed. by Per Christian Hansen, Jakob Jørgensen, and William R. B. Lionheart. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2021. DOI: 10.1137/1.9781611976670. eprint: https://epubs.siam.org/doi/pdf/10.1137/1.9781611976670. URL: https://epubs.siam.org/doi/abs/10.1137/1.9781611976670.

[4] Heinz H. Bauschke and Patrick L. Combettes. *Convex Analysis and Monotone Operator Theory in Hilbert Spaces*. 2nd ed. CMS Books in Mathematics. Springer Cham, 2017, pp. XIX+619. ISBN: 978-3-319-48310-8, 978-3-319-83911-0, 978-3-319-48311-5. DOI: 10.1007/978-3-319-48311-5. URL: https://doi.org/10.1007/978-3-319-48311-5.

[5] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. ISBN: 9780521833783. URL: https://web.stanford.edu/~boyd/cvxbook/.

[6] Antonin Chambolle and Thomas Pock. "A First-Order Primal-Dual Algorithm for Convex Problems with Applications to Imaging". In: *Journal of Mathematical Imaging and Vision* 40.1 (2011), pp. 120–145. ISSN: 1573-7683. DOI: 10.1007/s10851-010-0251-1. URL: https://doi.org/10.1007/s10851-010-0251-1.

[7] J. S. Jrgensen et al. "Core Imaging Library - Part I: a versatile Python framework for tomographic imaging". In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 379.2204 (July 2021), p. 20200192. ISSN: 1364-503X. DOI: 10.1098/rsta.2020.0192. eprint: https://royalsocietypublishing.org/rsta/article-pdf/doi/10.1098/rsta.2020.0192/1320532/rsta.2020.0192.pdf. URL: https://doi.org/10.1098/rsta.2020.0192.

[8] Hidetoshi Komiya. "Elementary proof for Sion's minimax theorem". In: *Kodai Mathematical Journal* 11.1 (1988), pp. 5–7. DOI: 10.2996/kmj/1138038812. URL: https://doi.org/10.2996/kmj/1138038812.